

react-dom/client의 역할

React가 정의한 화면을 브라우저의 실제 DOM에 표시하고 갱신하는 기능입니다.

① index.html: 화면이 들어갈 공간

```
<div id="root"></div>
```

② main.jsx: App을 root에 연결

createRoot: React가 관리할 루트를 생성

root.render: 그 루트에 App 화면을 표시

```
import { createRoot }  
  from "react-dom/client";  
import App from "./App.jsx";
```

```
const container =  
  document.getElementById("root");  
const root = createRoot(container);  
root.render(<App />);
```

react는 컴포넌트·상태 같은 공통 기능, react-dom/client는 브라우저 DOM 연결을 담당해요.

CSR: 브라우저에서 화면 만들기

서버가 전달한 JavaScript를 브라우저에서 실행해 게임 화면을 구성합니다.

처음 접속했을 때

1. 서버에서 HTML과 JavaScript를 받음
2. 브라우저가 main.jsx에서 시작한 JavaScript 코드를 실행
3. React가 App을 렌더링하고 root 안에 카드·점수판 DOM을 구성

카드를 터치했을 때

1. onClick으로 클릭 함수 실행
2. useState로 선택한 카드 번호 변경
3. React가 새 화면 결과를 계산하고 필요한 DOM 변경을 반영
4. 카드의 ?가 그림으로 바뀜

카드 선택마다 새 HTML 페이지를 받지 않아요. 휴대폰 브라우저에서도 같은 CSR 웹 앱이에요.

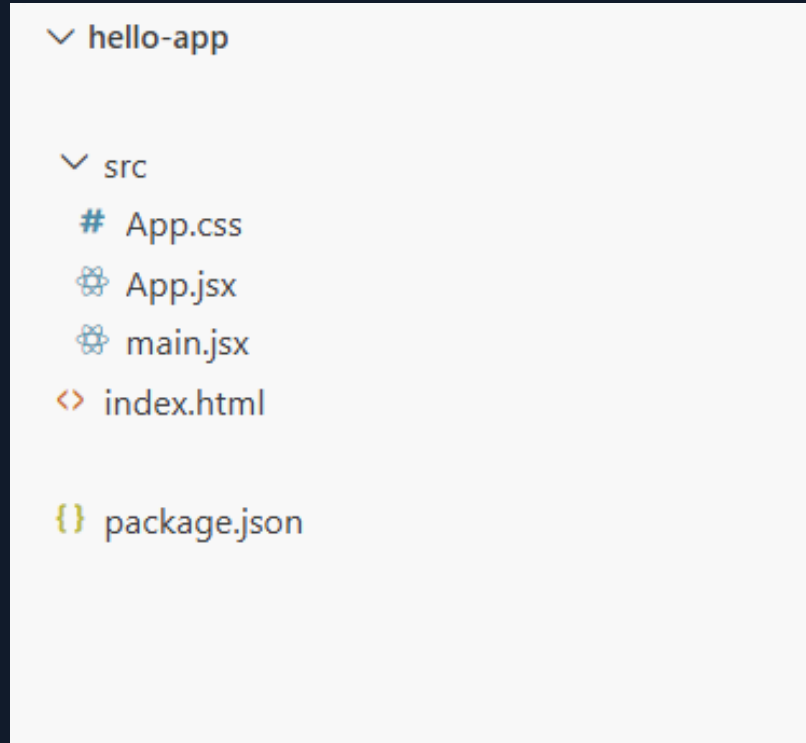
2_2 JSX 구문 이해하기

JSX는 JavaScript 코드 안에서 HTML과 비슷한 모양으로 화면을 작성하는 문법

JSX가 실제 화면이 되는 과정

브라우저는 JSX를 그대로 실행하지 못함.

프로젝트에서는 Vite가 JSX를 브라우저에서 실행할 수 있는 JavaScript로 변환



메모리 게임을 구성하는 5개 파일

파일	역할	이번 게임에서 하는 일
package.json	프로젝트 실행 설정	필요한 React·Vite와 npm run dev 실행 명령 관리
index.html	브라우저가 여는 웹페이지	게임 화면을 넣을 <div id="root"> 공간 준비
src/main.jsx	React와 웹페이지 연결	App을 root 공간에 표시
src/App.jsx	게임 화면과 동작	카드 표시, 클릭, 짝 비교, 점수, 다시 시작 처리
src/App.css	화면 디자인	배경색, 카드 크기, 배치, 버튼 모양 설정

실습에서는 App.jsx로 게임 기능을 만들고, App.css로 화면을 꾸며요.

03 package.json 입력

```
{  
  "name": "hello-app",  
  "version": "1.0.0",  
  "private": true,  
  "type": "module",  
  "scripts": {  
    "dev": "vite"  
  }  
}
```

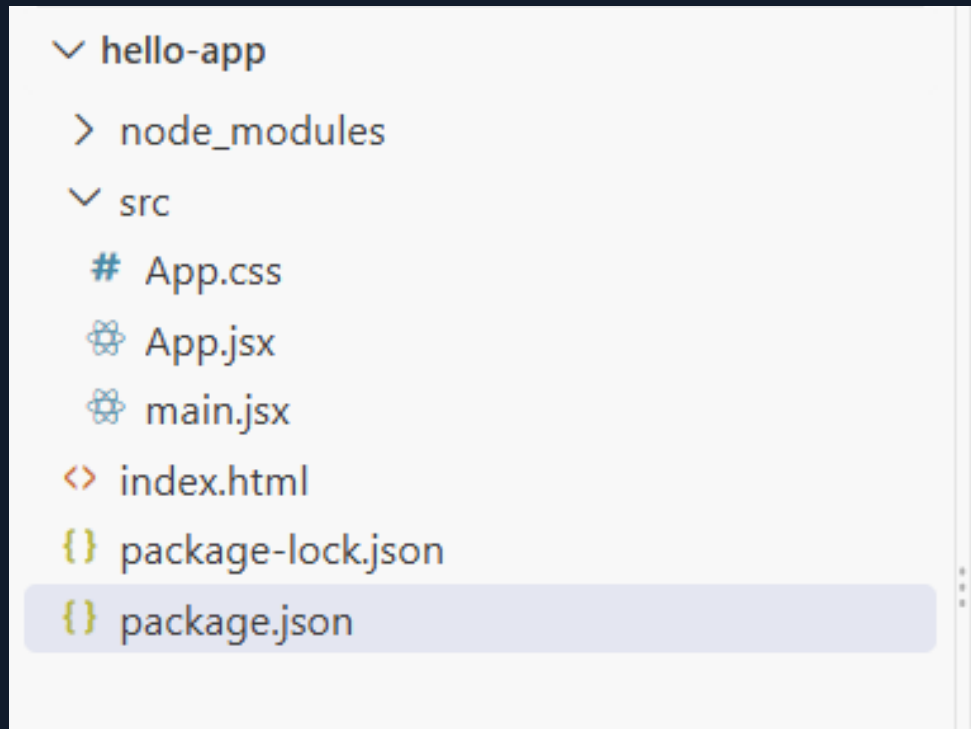
입력 후 Ctrl + S로 저장해요. JSON에는 주석을 넣지 않아요.

04 React와 Vite 설치

터미널 - 새터미널

npm install react react-dom

npm install -D vite



memory-game 터미널에서 한 줄씩 실행해요. 설치가 끝난 뒤 다음 명령을 입력해요.

설치 후 자동으로 생기는 파일

npm이 패키지를 내려받고, 설치 정보를 프로젝트에 기록해요.

항목	설치 후 변화	쉽게 이해하기
package.json	패키지 이름과 버전 범위 추가 React: dependencies Vite: devDependencies	준비물 목록
package-lock.json	정확한 설치 버전과 패키지 의존 관계 자동 기록	준비한 물품의 상세 명세
node_modules	React·Vite와 필요한 다른 패키지의 실제 코드 저장	실제 준비물

node_modules와 package-lock.json은 직접 작성하지 않아도 됩니다.

설치한 Vite로 게임 실행하기

① package.json에서 실행 설정 확인

```
"scripts": {  
  "dev": "vite"  
}
```

위 코드는 package.json의 일부예요. 기존 scripts 설정을 확인하세요.

② 같은 프로젝트 터미널에서 실행

```
npm run dev
```

dev에 연결된 vite가 실행돼요. 터미널의 Local 주소를 브라우저에서 여세요.

설치는 필요한 도구를 준비하는 작업, 실행은 개발 서버를 켜는 작업이에요.

JSX로 만드는 나의 첫 캐릭터 화면

프로젝트 준비부터 휴대폰 접속까지

Windows · VS Code · React + Vite

실습 1단계



버그 사냥꾼

Lv. 7 · 코딩 마법사

⚔️ 공격력: 80

🛡️ 방어력: 50

🔥 **전투력: 130**

❤️ 체력: 30 / 100



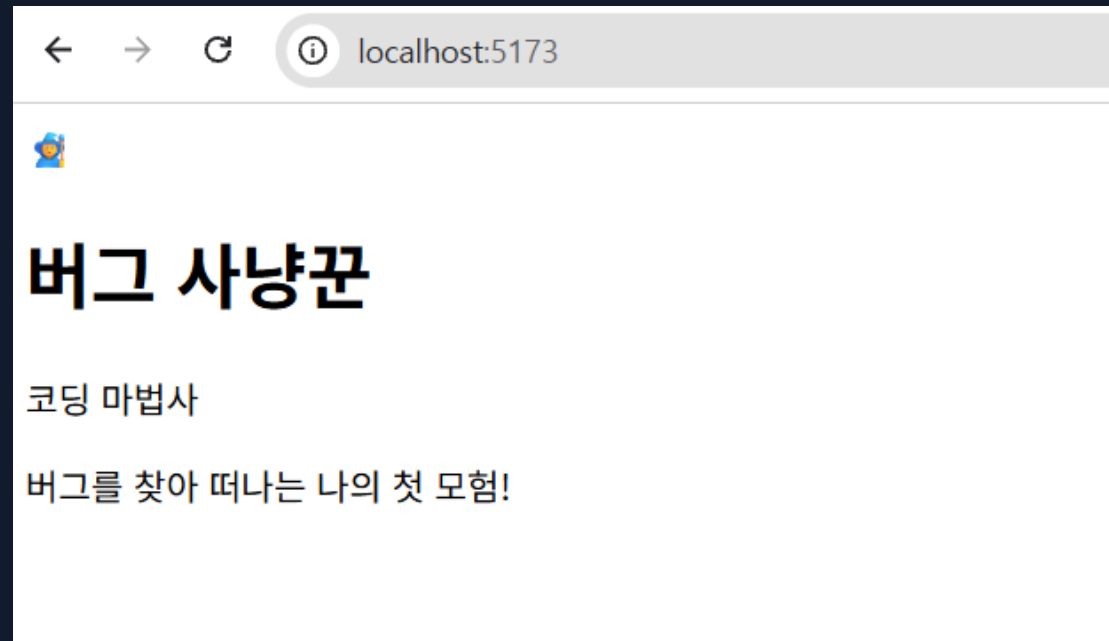
🔔 물약이 필요해요!

오늘의 완성 화면

캐릭터 이모지

이름과 직업

한 줄 소개



오늘은 JSX로 내용을 표시해요. 디자인과 능력치는 다음 실습에서 추가해요.

JSX의 역할

JavaScript 안에서
HTML처럼 화면 구조를 쓰는 문법

```
<h1>버그 사냥꾼</h1>  
<p>코딩 마법사</p>
```

태그를 보고 어떤 내용이 제목이고 문단인지 알아볼 수 있어요.

준비물과 시작 상태

1. 컴퓨터에 VS Code와 Node.js가 설치되어 있어요.
2. 설치 확인은 터미널에서 `node -v`, `npm -v`로 해요.
3. 휴대폰은 컴퓨터와 같은 Wi-Fi에 연결할 수 있어요.
4. 기존 `hello-app`이 있으면 해당 폴더를 사용해요.

버전 번호가 나오면 다음 단계로 진행해요.

01 프로젝트 폴더 열기

1. 파일 탐색기에서 hello-app 폴더를 만들어요.
2. VS Code의 파일 > 폴더 열기를 선택해요.
3. hello-app 폴더를 선택하고 열어요.

예시 위치: C:\Users\사용자이름\hello-app

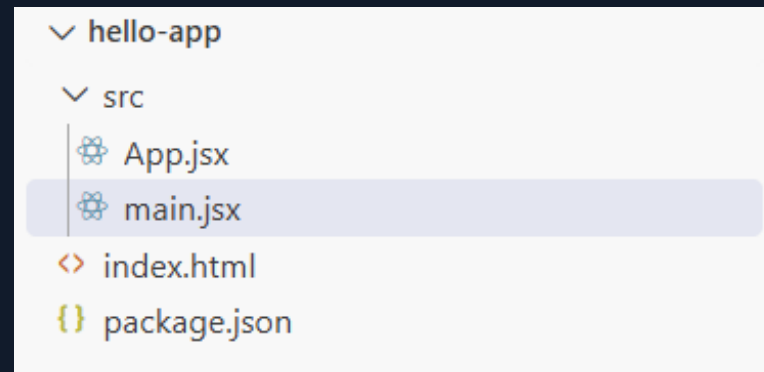
이미 만들어 둔 폴더가 있다면 새로 만들지 않고 열어요.

02 폴더와 파일 만들기

hello-app 안에 src 폴더를 만들어요.

src 안에 App.jsx와 main.jsx를 만들어요.

hello-app 바로 아래에 index.html과 package.json을 만
들어요.



탐색기에서 폴더를 오른쪽 클릭한 뒤 새 파일 또는 새 폴더를 선택해요.

03 package.json 작성

```
{  
  "name": "hello-app",  
  "version": "1.0.0",  
  "private": true,  
  "type": "module",  
  "scripts": {  
    "dev": "vite",  
    "build": "vite build",  
    "preview": "vite preview"  
  }  
}
```

프로젝트 최상위 파일

실행 명령을 등록해요.

입력 후 Ctrl + S

새 실습용 파일 기준이에요. JSON에는 주석을 넣지 않아요.

04 React와 Vite 설치

VS Code의 터미널 > 새 터미널을 열어요.

```
npm install react react-dom
```

설치가 끝나고 입력할 수 있는 줄이 다시 나타나면 실행해요.

```
npm install -D vite
```

터미널의 현재 폴더가 hello-app인지 확인해요. 두 명령은 한 줄씩 실행해요.

설치 완료 확인

node_modules 폴더가 자동으로 생겨요.

package-lock.json 파일이 자동으로 생겨요.

package.json에 설치한 패키지 정보가 추가돼요.

npm error가 나오면 오류를 해결한 뒤 진행해요.

자동 생성한 폴더와 파일은 직접 입력하거나 수정할 필요가 없어요.

05 App.jsx 작성

```
import React from "react";

export default function App() {
  return (
    <main>
      <div>#x1F9D9;</div>
      <h1>버그 사냥꾼</h1>
      <p>코딩 마법사</p>
      <p>버그를 찾아 떠나는 나의 첫 모험!</p>
    </main>
  );
}
```

#x1F9D9;는
마법사 이모지를
표시하는 코드예요.

위치: src/App.jsx · 화면에 표시할 내용을 작성하고 Ctrl + S로 저장해요.

App.jsx 코드 읽기

App 함수는 화면을 구성하는 컴포넌트예요.

return은 화면에 표시할 JSX를 반환해요.

<main>은 이번 화면의 내용을 하나로 묶어요.

<h1>은 제목, <p>는 문단을 표시해요.

태그는 <p>내용</p>처럼 열고 닫는 짝을 맞춰요.

06 main.jsx 작성

```
import React from "react";  
import { createRoot } from "react-dom/client";  
import App from "../App.jsx";
```

```
const root = createRoot(  
  document.getElementById("root")  
);
```

```
root.render(<App />);
```

위치: src/main.jsx · HTML의 root 자리에 App을 표시해요.

07 index.html 작성

```
<!DOCTYPE html>
<html lang="ko">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport"
      content="width=device-width, initial-scale=1.0" />
    <title>나의 캐릭터 카드</title>
  </head>
  <body>
    <div id="root"></div>
    <script type="module" src="/src/main.jsx"></script>
  </body>
</html>
```

위치: hello-app/index.html · src 폴더 바깥에 저장해요.

세 파일의 연결

index.html

root 자리를 만들고 main.jsx를 불러와요.

main.jsx

root 자리에 <App />을 표시해요.

App.jsx

캐릭터 이름과 소개를 JSX로 작성해요.

내용을 바꾸는 실습은 주로 App.jsx에서 진행해요.

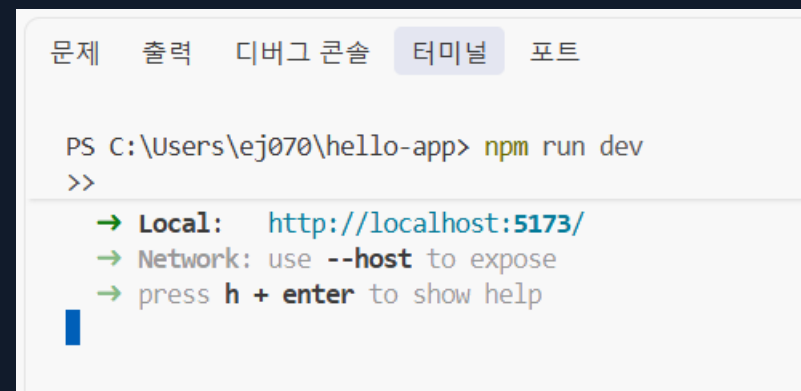
08 컴퓨터에서 실행

모든 파일을 저장한 뒤 hello-app 터미널에서 실행해요.

```
npm run dev
```

터미널에 나타난 Local 주소를 Ctrl + 클릭해요.

Local: <http://localhost:5173/>



```
문제  출력  디버그 콘솔  터미널  포트

PS C:\Users\ej070\hello-app> npm run dev
>>
→ Local:   http://localhost:5173/
→ Network: use --host to expose
→ press h + enter to show help
```

5173은 기본 포트예요. 다른 번호가 나오면 실제 표시된 주소를 사용해요.

09 내 캐릭터로 변경

App.jsx에서 네 가지 내용을 바꾸고 저장해요.

이모지	Windows 키 + 마침표로 선택
이름	버그 사냥꾼 대신 내가 정한 별명
직업	코딩 마법사 대신 새로운 직업
소개	캐릭터를 설명하는 한 문장

저장 후 브라우저를 확인해요. 화면이 그대로라면 새로고침해요.

10 휴대폰 연결 준비

1. 컴퓨터와 휴대폰을 같은 Wi-Fi에 연결해요.
2. 컴퓨터 터미널에서 Ctrl + C로 서버를 종료해요.
3. 아래 명령으로 서버를 다시 실행해요.

```
npm run dev -- --host 0.0.0.0
```

휴대폰으로 접속하는 동안 컴퓨터와 서버를 켜 두어요.

11 휴대폰에서 Network 주소 접속

Local: `http://localhost:5173/`

Network: `http://192.168.0.12:5173/`

휴대폰 크롬이나 사파리의 주소창을 열어요.

본인 터미널에 나온 Network 주소 전체를 입력해요.

`http://`와 포트 번호까지 확인해요.

192.168.0.12는 예시예요. 휴대폰에서 `localhost`를 입력하면 안 돼요.

```
문제  출력  디버그 콘솔  터미널  포트

PS C:\Users\ej070\hello-app> npm run dev -- --host 0.0.0.0
>> npm run dev -- --host 0.0.0.0
>>

VITE v8.3.1 ready in 242 ms

→ Local:   http://localhost:5173/
→ Network: http://172.30.1.24:5173/ Wi-Fi
→ press h + enter to show help
```

휴대폰 실행 성공

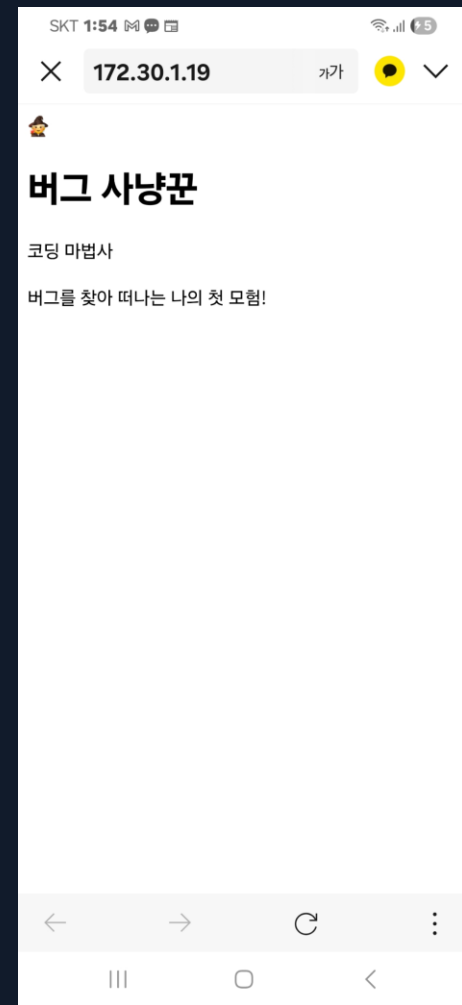
이름과 직업이 표시돼요.

소개 문장도 확인할 수 있어요.

컴퓨터에서 이름을 바꾸고 저장한 뒤

휴대폰에서도 변경을 확인해요.

화면이 그대로라면 휴대폰 브라우저를 새로고침해요.



05 App.jsx 전체 코드

```
import React from "react";
export default function App() {
  const nickname = "버그 사냥꾼";
  const job = "코딩 마법사";
  const attack = 80;
  const defense = 50;
  return (<main>
    <div>&#x1F9D9;</div>
    <h1>{nickname}</h1>
    <p>{job}</p>
    <p>버그를 찾아 떠나는 나의 첫 모험!</p>
    <hr />
    <p>공격력: {attack}</p>
    <p>방어력: {defense}</p>
    <h2>전투력: {attack + defense}</h2>
  </main>);
}
```

별명과 직업은 `const nickname`과 `const job`의 따옴표 안에서 바꿀 수 있어요.

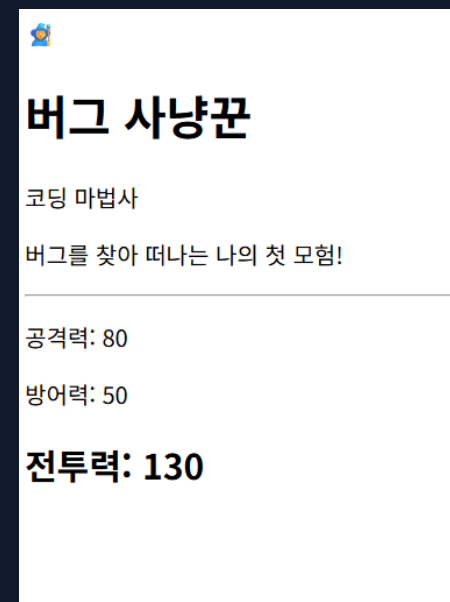
06 실행과 숫자 확인

```
npm run dev
```

터미널의 Local 주소를 Ctrl + 클릭해요.

공격력 80
방어력 50
전투력 130

이미 서버가 실행 중이라면 다시 실행하지 않고 열린 화면을 확인해요.



JSX의 변수와 계산식

`<p>공격력: {attack}</p>`

`<h2>전투력: {attack + defense}</h2>`

`{attack}`에는 변수 값 800이 표시돼요.

`{attack + defense}`에는 계산 결과 1300이 표시돼요.

태그는 화면 구조를, `{ }`는 JavaScript 값을 연결해요.

이제 계산 기능이 있는 화면에 CSS 디자인을 연결해요.

07 App.css 파일 만들기

왼쪽 탐색기에서 src 폴더를 오른쪽 클릭해요.

새 파일을 선택하고 App.css라고 입력해요.

App.jsx와 같은 src 폴더 안에 두어요.

App.css를 열어요. 새 파일은 내용이 비어 있어요.

파일 이름의 A는 대문자예요. 확장자는 .css로 입력해요.

08 App.jsx에 CSS 연결

App.jsx 맨 위를 다음 두 줄로 만들어요.

```
import React from "react";  
import "./App.css";
```

기존 import React 아래에 두 번째 줄만 추가해요.
그 아래 캐릭터 코드는 그대로 두고 저장해요.

App.css가 비어 있으면 아직 화면은 바뀌지 않아요. 다음 장부터 스타일을 입력해요.

09 CSS ① 전체 배경

```
* {  
  box-sizing: border-box;  
}  
body {  
  margin: 0;  
  min-height: 100vh;  
  padding: 32px 16px;  
  display: grid;  
  place-items: center;  
  background: radial-gradient(circle at top, #293859, #101827 65%);  
  color: #f1f5f9;  
  font-family: "Malgun Gothic", sans-serif;  
}
```

Ctrl + S 저장 후 확인: 남색 배경과 밝은 글자 색이 적용돼요.

10 CSS ② 카드 모양

```
#root {  
  width: 100%;  
}  
main {  
  width: 100%;  
  max-width: 380px;  
  margin: 0 auto;  
  padding: 32px 24px;  
  background: #1b273b;  
  border: 1px solid #384961;  
  border-radius: 28px;  
  text-align: center;  
  box-shadow: 0 20px 50px #00000050;  
}
```

Ctrl + S 저장 후 확인: 가운데에 모서리가 둥근 카드가 생겨요.

11 CSS ③ 캐릭터 이미지

```
main > div:first-child {  
  display: grid;  
  place-items: center;  
  width: 112px;  
  height: 112px;  
  margin: 0 auto 24px;  
  background: #2a3b55;  
  border: 2px solid #6ee7b7;  
  border-radius: 50%;  
  font-size: 64px;  
}
```

Ctrl + S 저장 후 확인: 캐릭터가 커지고 민트색 원형 테두리가 생겨요.

12 CSS ④ 이름과 직업

```
h1 {  
  margin: 0 0 12px;  
  font-size: 30px;  
  letter-spacing: -1px;  
  overflow-wrap: anywhere;  
}  
main > p:first-of-type {  
  margin: 0 0 20px;  
  color: #6ee7b7;  
  font-weight: bold;  
}
```

Ctrl + S 저장 후 확인: 이름은 크게, 직업은 민트색으로 표시돼요.

13 CSS ⑤ 문장과 구분선

```
p {  
  margin: 14px 0;  
  color: #d5deeb;  
  font-size: 16px;  
  line-height: 1.7;  
}  
hr {  
  margin: 24px 0;  
  border: 0;  
  border-top: 1px solid #384961;  
}
```

Ctrl + S 저장 후 확인: 문장 간격과 얇은 구분선이 정리돼요.

14 CSS ⑥ 전투력 강조

```
h2 {  
  margin: 24px 0 0;  
  padding: 18px 12px;  
  background: #6ee7b7;  
  border-radius: 16px;  
  color: #102a25;  
  font-size: 26px;  
}
```

Ctrl + S 저장 후 확인: 전투력 1300이 민트색 영역으로 강조돼요.

15 디자인 완성 확인

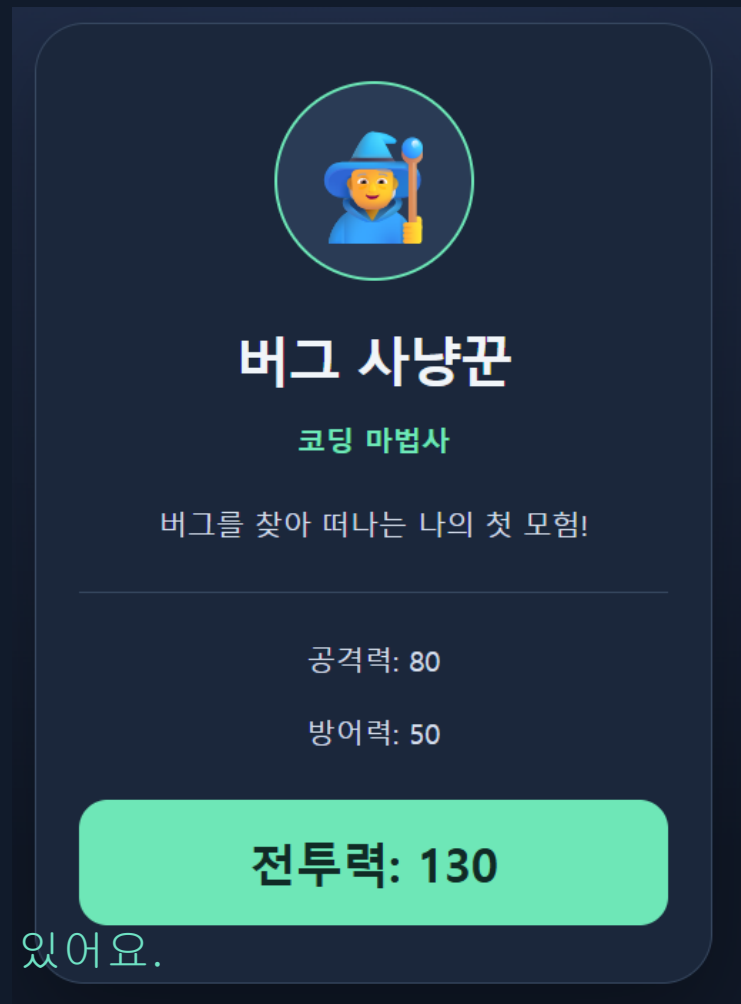
남색 배경과 둥근 카드

원형 테두리 안의 캐릭터

민트색 직업과 전투력 영역

공격력 80 + 방어력 50 = 130

오른쪽은 실제 완성 화면이에요. 기기에 따라 이모지 모양은 달라질 수 있어요.



휴대폰 실행 성공

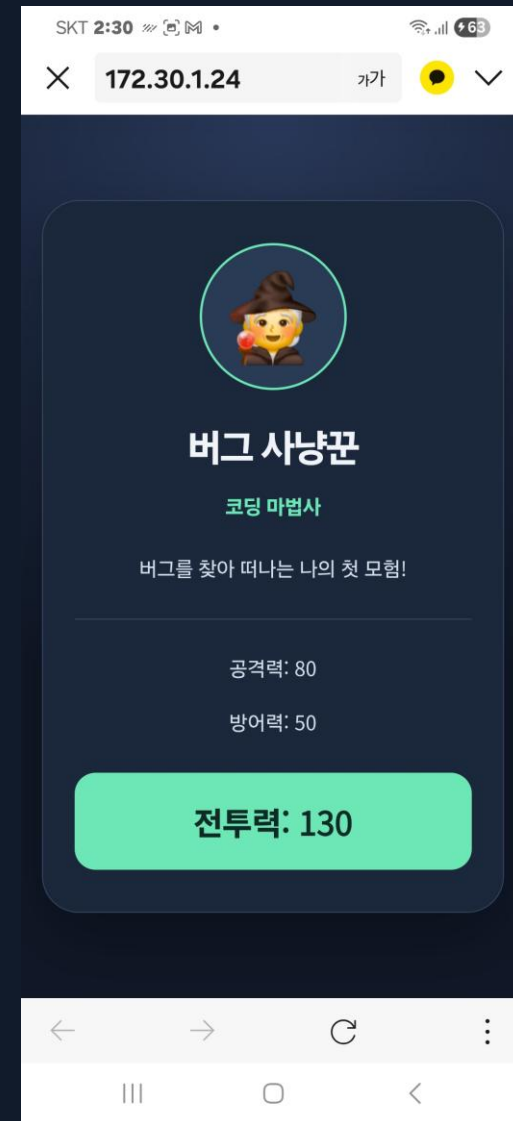
이름과 직업이 표시돼요.

소개 문장도 확인할 수 있어요.

컴퓨터에서 이름을 바꾸고 저장한 뒤

휴대폰에서도 변경을 확인해요.

화면이 그대로라면 휴대폰 브라우저를 새로고침해요.



선택 미션 전투력 200

```
const attack = 120;  
const defense = 80;
```

App.jsx에서 숫자 두 개만 바꾸고 저장해요.

디자인은 그대로, 전투력은 200으로 바뀌어요.

완성 예시와 같은 전투력 130으로 돌아가려면 80과 50으로 복원해요.

3단계 체력에 따른 경고 표시

기존 캐릭터 카드에 기능 추가하기

체력이 30 이하이면 경고 문구를 표시해요.

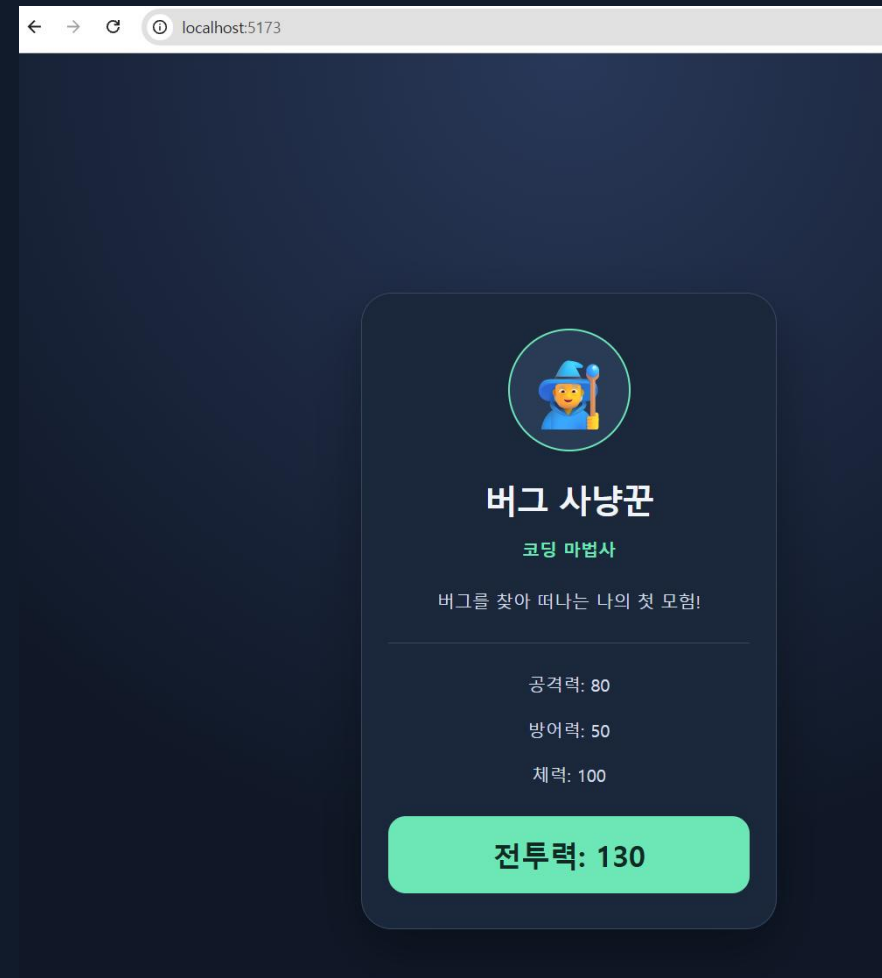
수정 파일: `src/App.jsx`와 `src/App.css`

현재 화면과 시작 위치

첨부 화면에는
체력 100이 표시되어 있어요.

지금 상태에서는
6쪽부터 이어서 진행해요.

체력이 없는 기준 파일로 시작하면 3쪽부터 따라 해요.



01 수정할 파일 열기

VS Code에서 hello-app 폴더를 열어요.

왼쪽 탐색기에서 src 폴더를 펼쳐요.

App.jsx를 클릭해요.

이번 실습은 지정한 위치에 코드를 추가해요.

이미 실행 중인 브라우저를 함께 열어 두어요. 각 단계 후 Ctrl + S로 저장해요.

src/App.jsx / defense 변수 바로 아래

02 체력 변수 한 줄 추가

찾을 코드

```
const defense = 50;
```

위 줄 바로 아래에 추가

```
const hp = 100;
```

이미 const hp가 있으면 다시 추가하지 않아요. 공격력과 방어력은 현재 값을 유지해요.

03 화면에 체력 한 줄 추가

찾을 코드

```
<p>방어력: {defense}</p>
```

추가 후 주변 코드

```
<p>방어력: {defense}</p>  
<p>체력: {hp}</p>  
<h2>전투력: {attack + defense}</h2>
```

저장 후 체력: 100이 보이면 성공이에요. 이미 있으면 중복으로 추가하지 않아요.

04 체력 아래에 경고 코드 추가

<p>체력: {hp}</p> 바로 아래에 다음 코드를 추가해요.

```
{hp <= 30 && (  
  <p className="warning">  
    &#x1F6A8; 위험! 체력을 회복하세요!  
  </p>  
)}
```

현재 hp는 100이므로 저장해도 경고가 보이지 않는 것이 정상이에요.

05 추가 위치 확인

```
<p>공격력: {attack}</p>
<p>방어력: {defense}</p>
<p>체력: {hp}</p>
{hp <= 30 && (
  <p className="warning">
    &#x1F6A8; 위험! 체력을 회복하세요!
  </p>
)}
<h2>전투력: {attack + defense}</h2>
```

이 코드는 위치 확인용이에요. 앞 단계에서 추가했다면 다시 붙여 넣지 않아요.

06 경고 문구 디자인 추가

```
.warning {  
  padding: 12px;  
  background: #4a2230;  
  border: 1px solid #fb7185;  
  border-radius: 12px;  
  color: #fda4af;  
  font-weight: bold;  
}
```

맨 아래에 추가하고 저장해요. 경고가 나타나면 붉은 배경과 테두리를 볼 수 있어요.

07 체력 20으로 위급 상황 만들기

수정 전

```
const hp = 100;
```

수정 후

```
const hp = 20;
```

Ctrl + S로 저장한 다음 브라우저를 확인해요. const hp 줄을 새로 추가하지 않아요.

08 실행 결과 확인

체력: 20

위험! 체력을 회복하세요!

체력 아래에 붉은 경고 상자가 나타나야 해요.
전투력은 기존 계산 결과를 유지해요.

경고가 없다면 hp 값, <= 기호, 저장 여부를 먼저 확인해요.



버그 사냥꾼
코딩 마법사

버그를 찾아 떠나는 나의 첫 모험!

공격력: 80
방어력: 50
체력: 20

🚨 위험! 체력을 회복하세요!

전투력: 130

09 체력 회복 후 경고 확인

```
const hp = 80;
```

저장 후 체력: 80이 표시돼요.

경고 문구와 붉은 상자가 사라져요.

20과 80을 번갈아 입력하며 결과를 확인해요.

값을 직접 수정하고 저장하는 실습이에요. 회복 버튼은 아직 만들지 않아요.



캐릭터 카드 완성 코드

App.jsx와 App.css 수정 실습

레벨과 전투력, 체력 막대와 물약 경고



기존 hello-app 프로젝트에서 두 파일을 교체해요.

파일 교체 순서

1. src/App.jsx를 열고 기존 내용을 모두 지워요.

2. src/App.css를 열고 기존 내용을 모두 지워요.

App.jsx는 세 부분을 모두 입력한 뒤 저장해요. 중간에는 태그가 닫히지 않아 오류가 보여요.

App.jsx 1/3 변수 준비

```
import React from "react";  
import "./App.css";  
  
export default function App() {  
  const nickname = "버그 사냥꾼";  
  const job = "코딩 마법사";  
  const level = 7;  
  const attack = 80;  
  const defense = 50;  
  const hp = 30;  
  const maxHp = 100;
```

🧙 같은 코드는 이모지를 나타내요. 다음 장도 같은 파일에 이어 붙여요.

App.jsx 2/3 캐릭터와 능력치

```
return (  
  <main className="character-card">  
    <div className="avatar">&#x1F9D9;</div>  
    <h1>{nickname}</h1>  
    <p className="job">Lv. {level} · {job}</p>  
    <hr />  
    <p>&#x2694;&#xFE0F; 공격력: {attack}</p>  
    <p>&#x1F6E1;&#xFE0F; 방어력: {defense}</p>  
    <h2>&#x1F525; 전투력: {attack + defense}</h2>  
    <p>&#x1F497; 체력: {hp} / {maxHp}</p>  
  </main>  
)
```

전투력은 공격력 + 방어력으로 계산해요. 다음 장에서 태그와 함수를 달아요.

App.jsx 3/3 체력 막대와 경고

```
<div className="hp-bar">
  <div
    className="hp-fill"
    style={{ width: `${(hp / maxHp) * 100}%` }}
  />
</div>
{hp <= 30 && (
  <p className="warning">⚠️; 물약이 필요해요!</p>
)}
</main>
);
}
```

여기까지 입력한 뒤 Ctrl + S로 저장해요. 코드의 ` 기호는 백틱이에요.

App.css 1/6 전체 배경

```
* {  
  box-sizing: border-box;  
}  
body {  
  margin: 0;  
  padding: 12px;  
  background: white;  
  color: white;  
  font-family: "Malgun Gothic", sans-serif;  
}  
#root {  
  width: 100%;  
}
```

이 코드를 입력한 뒤 다음 장을 같은 파일에 이어 붙여요.

App.css 2/6 카드와 캐릭터

```
.character-card {  
  width: 100%;  
  max-width: 490px;  
  margin: 0 auto;  
  padding: 48px 38px;  
  border-radius: 32px;  
  background: #192436;  
  text-align: center;  
}  
.avatar {  
  margin-bottom: 32px;  
  font-size: 90px;  
  line-height: 1.3;  
}
```

이 코드를 입력한 뒤 다음 장을 같은 파일에 이어 붙여요.

App.css 3/6 이름과 설명

```
.character-card h1 {  
  margin: 0 0 34px;  
  font-size: 40px;  
  letter-spacing: -1px;  
}  
.character-card p {  
  margin: 20px 0;  
  font-size: 22px;  
  line-height: 1.5;  
}  
.character-card .job {  
  margin-bottom: 20px;  
}
```

이 코드를 입력한 뒤 다음 장을 같은 파일에 이어 붙여요.

App.css 4/6 구분선과 전투력

```
.character-card hr {  
  margin: 20px 0;  
  border: 0;  
  border-top: 1px solid #999;  
}  
.character-card h2 {  
  margin: 28px 0;  
  font-size: 34px;  
}
```

이 코드를 입력한 뒤 다음 장을 같은 파일에 이어 붙여요.

App.css 5/6 체력 막대

```
.hp-bar {  
  width: 220px;  
  max-width: 100%;  
  height: 22px;  
  margin: 24px auto 28px;  
  background: #808080;  
  overflow: hidden;  
}  
.hp-fill {  
  height: 100%;  
  background: #008000;  
}
```

이 코드를 입력한 뒤 다음 장을 같은 파일에 이어 붙여요.

App.css 6/6 경고와 작은 화면

```
.character-card .warning {  
  margin: 28px 0 10px;  
}  
@media (max-width: 400px) {  
  .character-card {  
    padding: 36px 20px;  
  }  
  .character-card h1 {  
    font-size: 32px;  
  }  
  .character-card h2 {  
    font-size: 28px;  
  }  
}
```

CSS 여섯 부분을 모두 입력했으면 Ctrl + S로 저장해요.